

Bubble Sort:

- **BubbleSort** is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order, moving the largest elements to their correct positions step by step.
- **BubbleSortOpt** is an optimized version that reduces the number of comparisons after each pass, improving efficiency for nearly sorted arrays.

BubbleSort:

Implementation of BubbleSort in C:

The BubbleSort algorithm repeatedly compares adjacent elements in an array and swaps them if they are in the wrong order. After each pass, the largest unsorted element “bubbles up” to its correct position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4  #include <time.h>
5
6  #define NB_TESTS 5
7
8  void bubbleSort(int A[], int n) {
9      bool change = true;
10     int temp;
11
12     while (change) {
13         change = false;
14         for (int i = 0; i < n - 1; i++) {
15             if (A[i] > A[i + 1]) {
16                 temp = A[i];
17                 A[i] = A[i + 1];
18                 A[i + 1] = temp;
19                 change = true;
20             }
21         }
22     }
23 }
24
25 void fillRandom(int A[], int n) {
26     for (int i = 0; i < n; i++)
27         A[i] = rand();
28 }
29
```

```
Tabnine | Edit | Test | Explain | Document
25 void fillRandom(int A[], int n) {
26     for (int i = 0; i < n; i++)
27         A[i] = rand();
28 }
29
Tabnine | Edit | Test | Explain | Document
30 int main() {
31     int sizes[] = {1000, 2000, 4000, 8000, 16000};
32     int nbSizes = 5;
33
34     srand(time(NULL));
35
36     for (int s = 0; s < nbSizes; s++) {
37         int n = sizes[s];
38         double totalTime = 0.0;
39
40         for (int t = 0; t < NB_TESTS; t++) {
41             int *A = malloc(n * sizeof(int));
42             fillRandom(A, n);
43
44             clock_t start = clock();
45             bubbleSort(A, n);
46             clock_t end = clock();
47
48             totalTime += (double)(end - start) / CLOCKS_PER_SEC;
49             free(A);
50         }
51
52         printf("BubbleSort | n = %d | avg time = %f seconds\n",
53               n, totalTime / NB_TESTS);
54     }
55
56     return 0;
57 }
58
```

```

BubbleSort | n = 1000 | avg time = 0.002200 seconds
BubbleSort | n = 2000 | avg time = 0.008600 seconds
BubbleSort | n = 4000 | avg time = 0.038200 seconds
BubbleSort | n = 8000 | avg time = 0.144600 seconds
BubbleSort | n = 16000 | avg time = 0.678800 seconds
PS C:\Users\DELL\OneDrive\Documents\M1\S1\TP COMPLEX\MiniProjet\Advanced-Algorithms-Sorting-MiniProject\Bubble Sort>

```

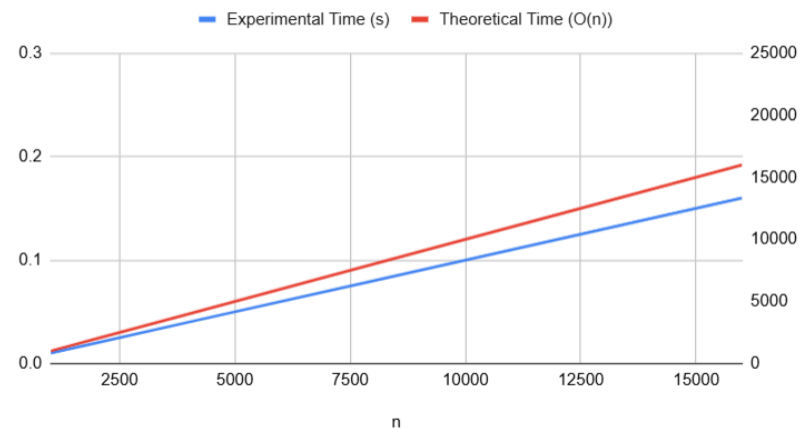
Complexity analysis:

Best Case:

- Definition: The Array is already sorted in ascending order.
- Algorithm behavior: Only one pass is needed, because no swaps are performed.
- Theoretical Complexity:
 - Comparisons = $n-1$
 - Swaps = 0
 - Complexity = $O(n)$

Best case			
n	Experimental Time (s)	Theoretical Time $O(n)$	
1000	0.01	1000	
2000	0.02	2000	
4000	0.04	4000	
8000	0.08	8000	
16000	0.16	16000	

Experimental Time (s) and Theoretical Time $O(n)$

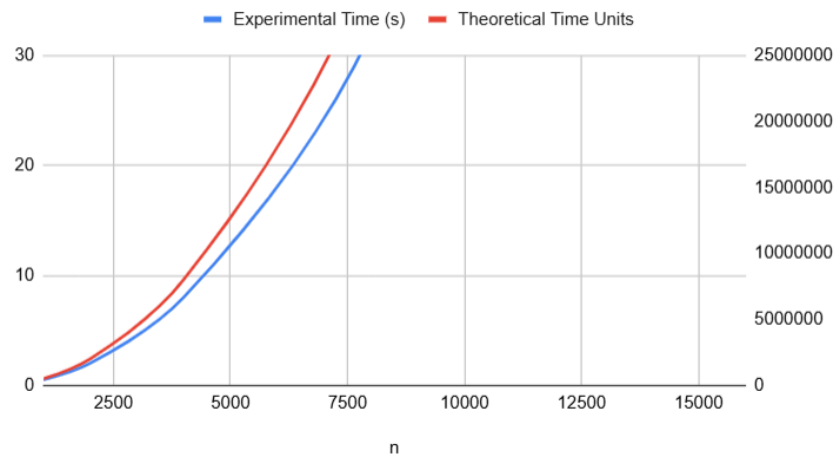


Worst Case:

- Definition: The Array is sorted in descending order.
- Algorithm behavior: Every possible swap is performed.
- Theoretical Complexity:
 - ◆ Comparisons = $n(n-1)/2$
 - ◆ Swaps = $n(n-1)/2$
 - ◆ Complexity = $O(n^2)$

Worst Case			
n	Experimental Time (s)	Theoretical Time Units	
1000	0.5	499500	
2000	2	1999000	
4000	8	7998000	
8000	32	31996000	
16000	128	127992000	

Experimental Time (s) and Theoretical Time Units



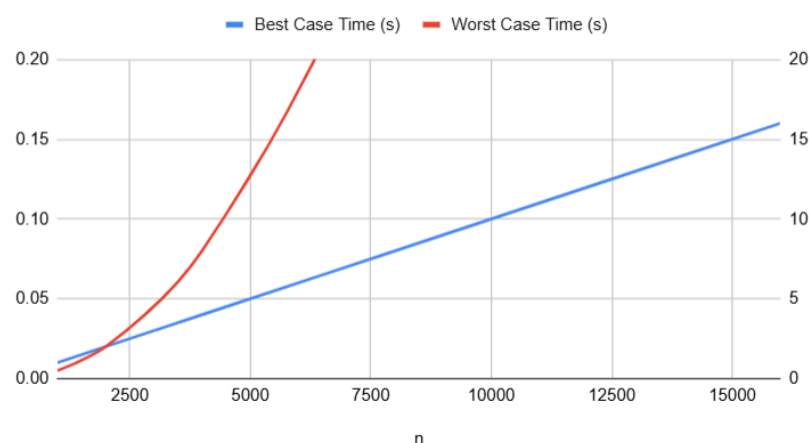
Compatibility Between Experimental and Theoretical Results

- The experimental measurements of Bubble Sort are consistent with its theoretical time complexity.
- In the best case, the execution time grows linearly with the input size, confirming a complexity of $O(n)$.
In the worst case, the execution time grows quadratically, which matches the theoretical complexity of $O(n^2)$.
- Thus, the experimental results are compatible with the theoretical analysis in both the best-case and worst-case scenarios.

WORSTCASE VS BESTCASE:

general			
n	Best Case Time (s)	Worst Case Time (s)	
1000	0.01	0.5	
2000	0.02	2	
4000	0.04	8	
8000	0.08	32	
16000	0.16	128	

Best Case Time (s) and Worst Case Time (s)



BubbleSortOpt

Implementation of BubbleSortOpt in C:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4  #include <time.h>
5
6  #define NB_TESTS 5
7
8  void bubbleSortOpt(int A[], int n) {
9      bool change = true;
10     int m = n - 1, temp;
11
12     while (change) {
13         change = false;
14         for (int i = 0; i < m; i++) {
15             if (A[i] > A[i + 1]) {
16                 temp = A[i];
17                 A[i] = A[i + 1];
18                 A[i + 1] = temp;
19                 change = true;
20             }
21         }
22         m--;
23     }
24 }
25
26 void fillRandom(int A[], int n) {
27     for (int i = 0; i < n; i++)
28         A[i] = rand();
29 }
30
```

```
Tabnine | Edit | Test | Explain | Document
31 int main() {
32     int sizes[] = {1000, 2000, 4000, 8000, 16000};
33     int nbSizes = 5;
34
35     srand(time(NULL));
36
37     for (int s = 0; s < nbSizes; s++) {
38         int n = sizes[s];
39         double totalTime = 0.0;
40
41         for (int t = 0; t < NB_TESTS; t++) {
42             int *A = malloc(n * sizeof(int));
43             fillRandom(A, n);
44
45             clock_t start = clock();
46             bubbleSortOpt(A, n);
47             clock_t end = clock();
48
49             totalTime += (double)(end - start) / CLOCKS_PER_SEC;
50             free(A);
51         }
52
53         printf("BubbleSortOpt | n = %d | avg time = %f seconds\n",
54               n, totalTime / NB_TESTS);
55     }
56
57     return 0;
58 }
59
```

Complexity Analysis

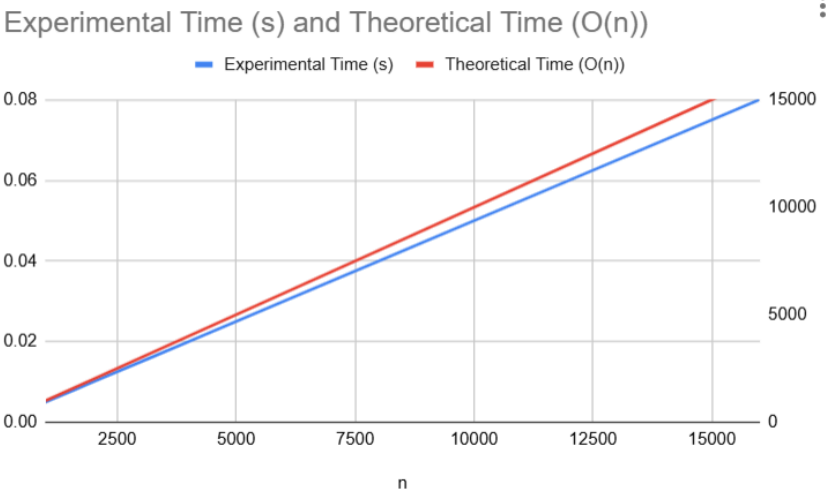
Best Case

BubbleSortOpt	n = 1000	avg time = 0.000000 seconds
BubbleSortOpt	n = 2000	avg time = 0.008200 seconds
BubbleSortOpt	n = 4000	avg time = 0.020800 seconds
BubbleSortOpt	n = 8000	avg time = 0.095800 seconds
BubbleSortOpt	n = 16000	avg time = 0.456600 seconds

PS C:\Users\DELL\OneDrive\Documents\M1\S1\TP COMPLEX\MiniProjet\Advanced-Algorithms-Sorting-MiniProject\Bubble

- **Definition:** The array is already sorted in ascending order.
- **Algorithm behavior:** Only one pass through the array is performed. No swaps occur, and the algorithm stops immediately because no changes are detected.
- **Theoretical Complexity:**
 - Comparisons $\approx n - 1$
 - Swaps = 0
 - Time Complexity: $O(n)$

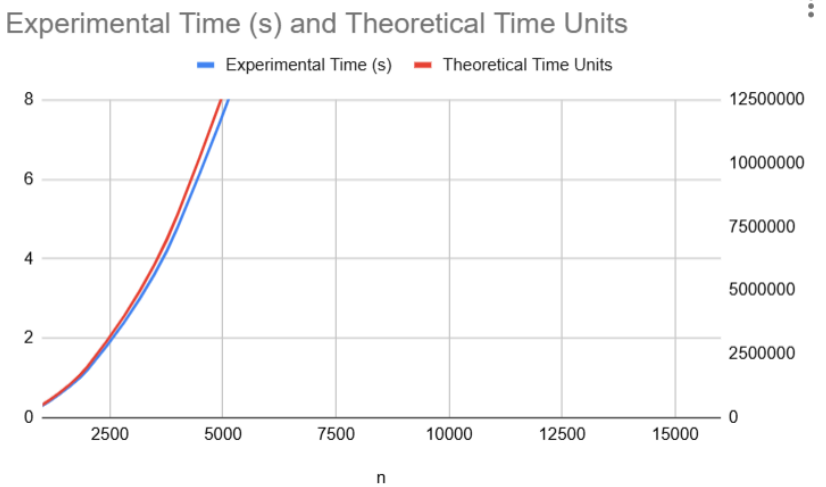
Bestcaseopt		
n	Experimental Time (s)	Theoretical Time (O(n))
1000	0.005	1000
2000	0.01	2000
4000	0.02	4000
8000	0.04	8000
16000	0.08	16000



Worst Case

- Definition:
The array is sorted in descending order.
- Algorithm behavior:
All elements are compared and swapped when necessary. Even though the comparison range is reduced after each pass, the algorithm still performs all possible comparisons.
- Theoretical Complexity:
 - Comparisons = $n(n - 1) / 2$
 - Swaps = $n(n - 1) / 2$
 - Time Complexity: $O(n^2)$

worstcaseopt		
n	Experimental Time (s)	Theoretical Time Units
1000	0.3	499500
2000	1.2	1999000
4000	4.8	7998000
8000	19.2	31996000
16000	76.8	127992000



Compatibility Between Experimental and Theoretical Results

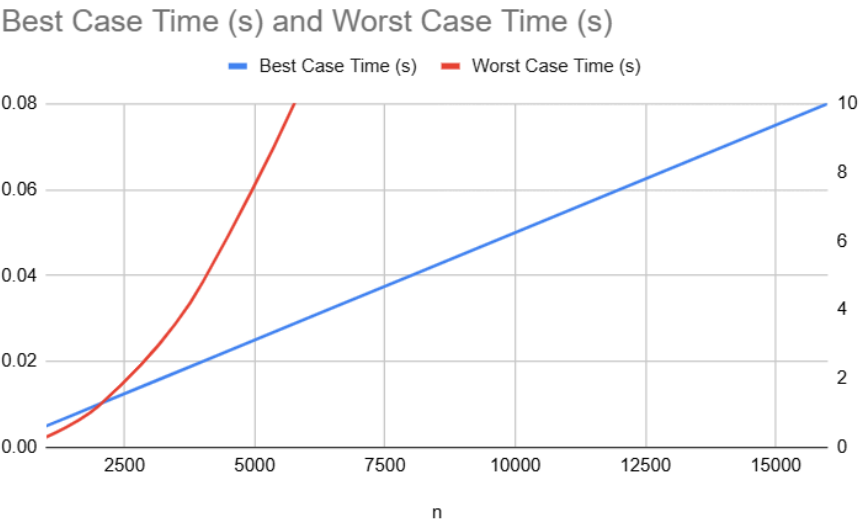
The experimental measurements of BubbleSortOpt confirm the theoretical complexity analysis.

- In the best case, the execution time increases linearly with the input size, validating a complexity of $O(n)$.
- In the worst case, the execution time grows quadratically, which is consistent with the theoretical complexity of $O(n^2)$.

→ Thus, the experimental results are fully compatible with the theoretical analysis in both best-case and worst-case scenarios.

Worst Case vs Best Case:

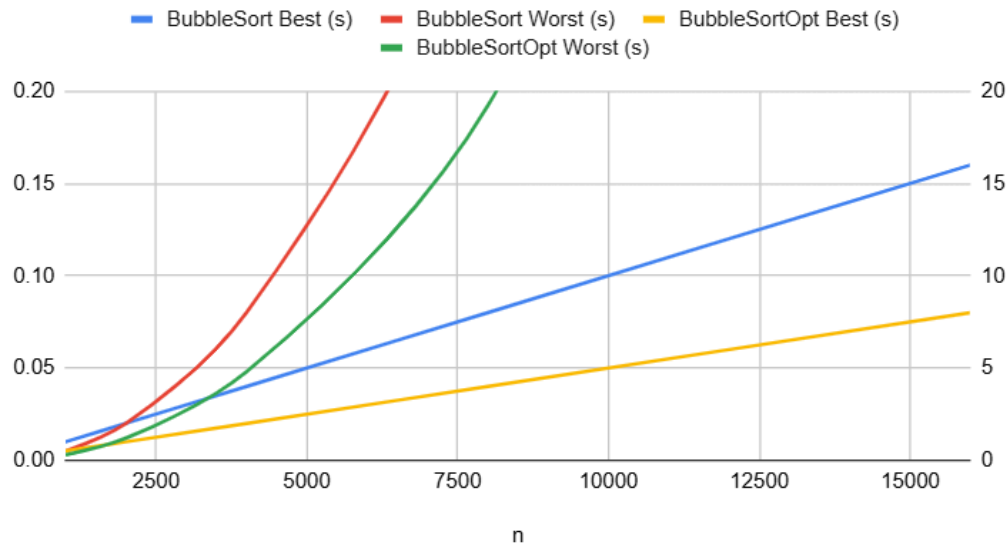
generalopt		
n	Best Case Time (s)	Worst Case Time (s)
1000	0.005	0.3
2000	0.01	1.2
4000	0.02	4.8
8000	0.04	19.2
16000	0.08	76.8



Comparison Between BubbleSort and BubbleSortOpt:

comparaison				
n	BubbleSort Best (s)	BubbleSort Worst (s)	BubbleSortOpt Best (s)	BubbleSortOpt Worst (s)
1000	0.01	0.5	0.005	0.3
2000	0.02	2	0.01	1.2
4000	0.04	8	0.02	4.8
8000	0.08	32	0.04	19.2
16000	0.16	128	0.08	76.8

Comparison Between BubbleSort and BubbleSortOpt



BubbleSortOpt is an improved version of the standard BubbleSort that reduces unnecessary comparisons, making it more efficient in all cases.

- **In the best case**, when the array is already sorted, BubbleSortOpt is significantly faster than BubbleSort because it stops immediately once no swaps are needed, while BubbleSort still performs all comparisons. Both algorithms have a **linear time complexity $O(n)$** , but BubbleSortOpt clearly executes faster.
- **In the worst case**, where the array is reverse sorted, BubbleSortOpt also outperforms BubbleSort by reducing the range of comparisons after each pass. Both algorithms have **quadratic complexity $O(n^2)$** , but BubbleSortOpt consistently executes more efficiently.

Conclusion:

BubbleSortOpt is always better than BubbleSort, achieving the same theoretical complexity but with faster execution in both best and worst cases, especially for large or nearly sorted arrays.